

모두를 위한 딥러닝 (이미지처리편)

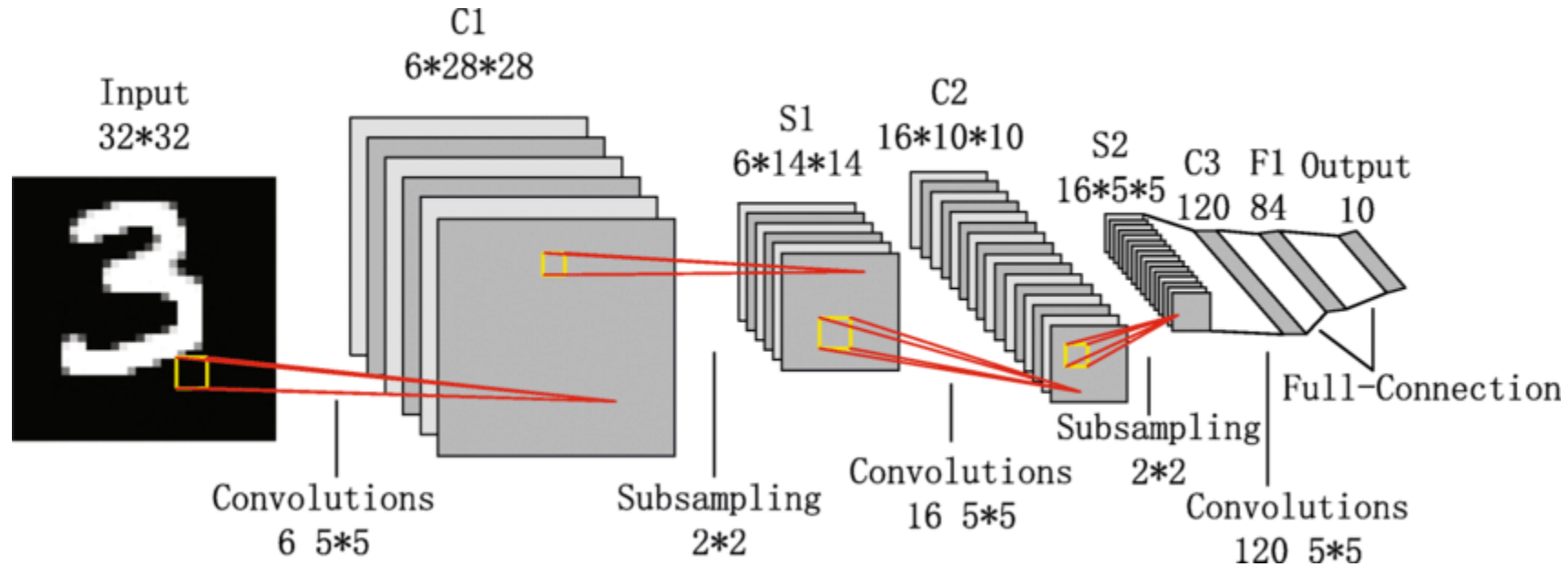
경북대학교 IT교육센터

허 찬

차시(모듈)	차시별 주제	학습내용
1	딥러닝을 이용한 이미지 처리	이미지 처리의 이해 다양한 이미지처리의 응용 분야에 대한 이해
2	특징 추출의 이해	이미지 데이터 전처리 기법의 이해 이미지 특징 추출의 이해 및 실습 딥러닝을 이용한 학습 및 추론 과정의 이해
3	Convolution Neural Network	합성곱 신경망의 이해
4	CNN 모델의 이해	다양한 CNN 모델의 이해 딥러닝의 여러가지 요소 이해
5	CNN을 이용한 분류 모델 실습	CNN을 이용한 이미지 분류 모델 실습
6	전이 학습 모델의 이해	전이 학습의 이해 전이학습을 이용한 이미지 처리 모델 실습
7	객체 인식 모델의 이해	다양한 객체 인식 모델의 이해 Semantic segmentation (U-Net)의 이해
8	Transformer 기반 이미지 처리 모델 1	Attention과 Transformer의 이해
9	Transformer 기반 이미지 처리 모델 2	Transformer 기반 이미지 처리 모델 (ViT)의 이해
10	이미지 생성 모델의 이해	최신 이미지 생성 모델의 이해 오토인코더의 이해 및 실습
11	최신 이미지 처리 기술의 이해	최신 컴퓨터비전 모델의 이해 비디오 처리의 이해

4.1 다양한 CNN 모델의 이해

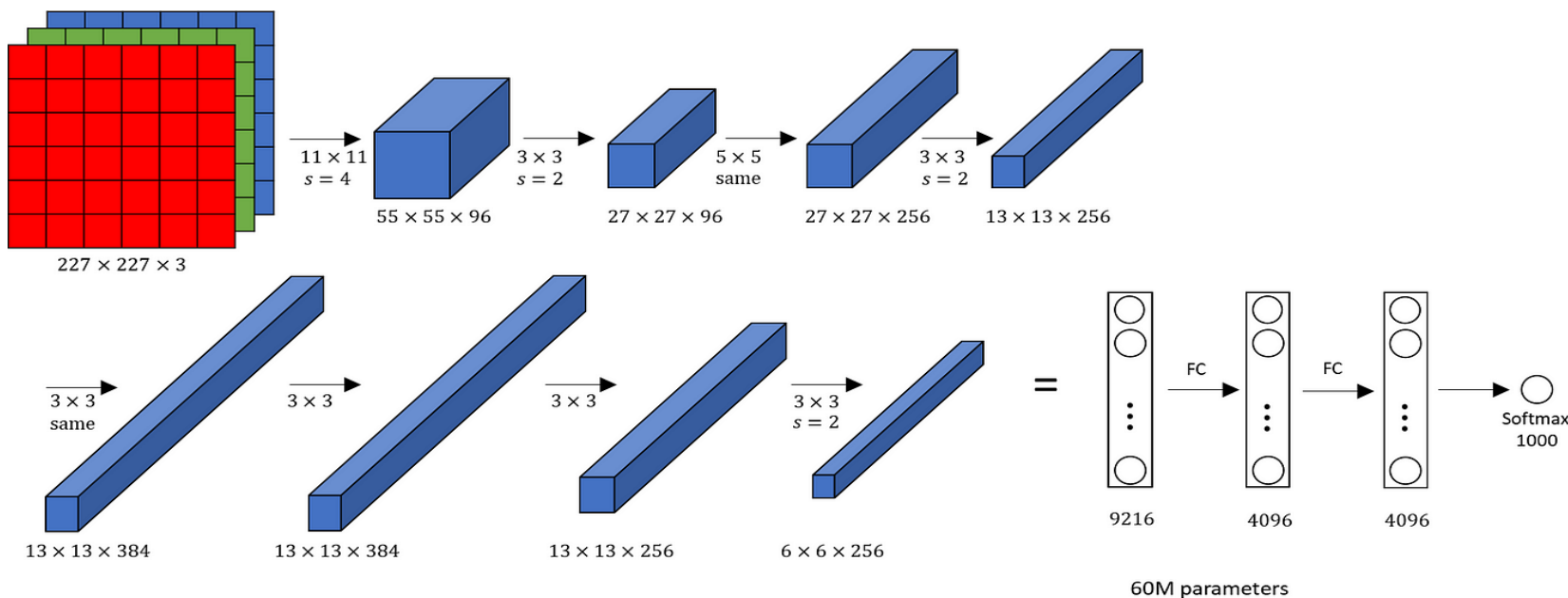
LeNet-5



- 도입년도: 1998년
- 특징: CNN의 초기 모델 중 하나로, 주로 손글씨 숫자 인식에 사용. 간단한 구조로, 몇 개의 컨볼루션 레이어와 서브샘플링(풀링) 레이어, 그리고 완전 연결 레이어로 구성.
- 응용: 숫자 인식, 기본적인 이미지 분류

4.1 다양한 CNN 모델의 이해

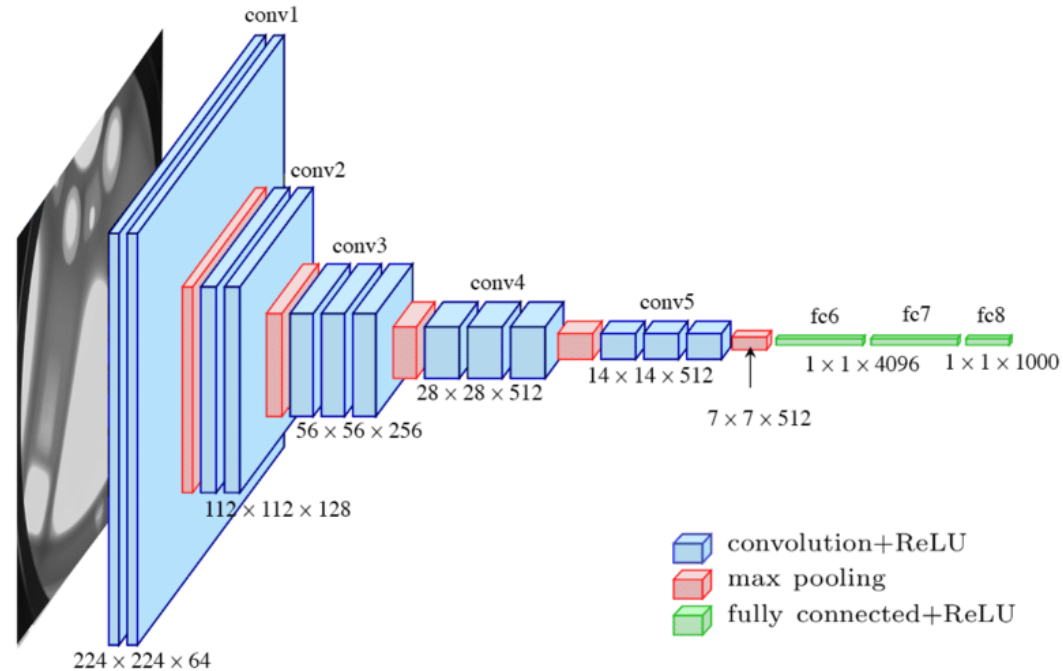
AlexNet



- 도입년도: 2012년
- 특징: 딥러닝의 성공을 이끈 획기적인 모델로, ImageNet 대회에서 우승하며 주목. LeNet과 유사하지만 더 깊고, ReLU 활성화 함수를 사용하며, 드롭아웃과 데이터 증강을 통한 과적합 방지 기법을 도입.
- 응용: 복잡한 이미지 분류, 패턴 인식

4.1 다양한 CNN 모델의 이해

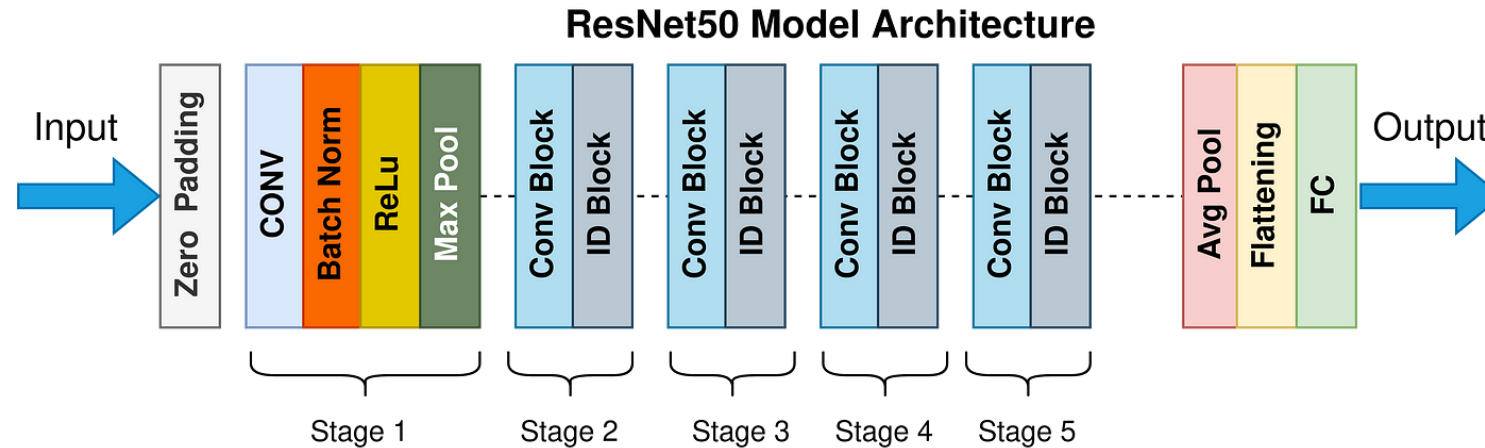
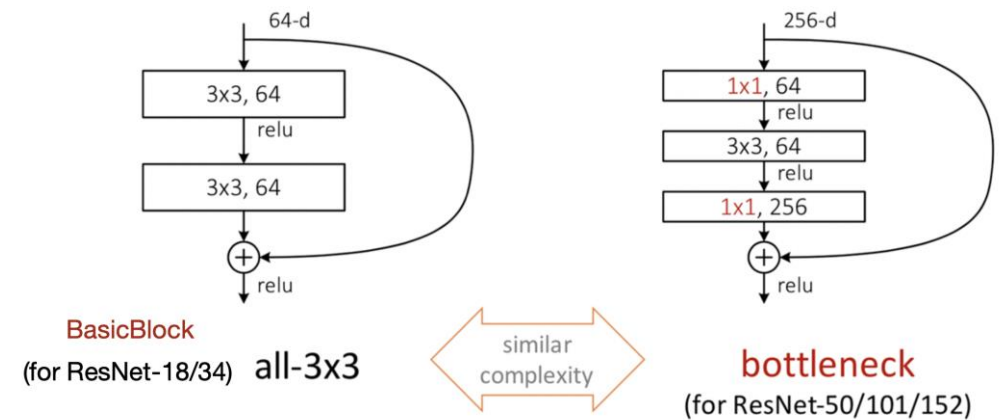
VGGNet



- 도입년도: 2014년
- 특징: 일관된 컨볼루션 필터(3×3)와 최대 풀링을 사용하는 구조로, 모델의 깊이에 따라 VGG16과 VGG19로 나뉨. VGGNet의 깊은 구조는 특징 추출에서 높은 성능을 보여줌.
- 응용: 이미지 분류, 특징 추출, 전이 학습

4.1 다양한 CNN 모델의 이해

ResNet



- 도입년도: 2015년
- 특징: 잔차 블록을 도입하여 깊은 네트워크에서도 학습이 잘 이루어지게 하며, 이는 네트워크가 깊어 질수록 발생할 수 있는 소실되는 기울기 문제를 완화. ResNet은 깊이가 152 레이어에 이르는 매우 깊은 모델을 성공적으로 학습.
- 응용: 매우 깊은 네트워크의 학습, 광범위한 이미지 및 비디오 분석 작업

딥러닝을 제대로 이해하려면...

Method	Description	Merits	Demerits
Back propagation	Used in Optimization problem	For calculation of gradient	Sensitive to noisy data
Stochastic Gradient Descent	To find optimal minimum in optimization problems	Avoids trapping in local minimum	Longer convergence time, computationally expensive
Learning Rate Decay	Reduce learning rate gradually	Increases performance, Reduces training time	Computationally expensive
Dropout	Dropsout units/ connection during training	Avoids overfitting	Increases number of iterations required to converge
Max-Pooling	Applies a max filter	Reduces dimension and computational cost	Considers only the maximum element which may lead to unacceptable result in some cases
Batch Normalization	Batch-wise normalization of input to a layer	Reduces covariant shift, Increases stability of the network, Network trains faster, Allows higher learning rates	Computational overhead during training
Skip-gram	Used in word embedding algorithms	Can work on any raw text, Requires less memory	Softmax function is computationally expensive, Training Time is high
Transfer learning	Knowledge of first model is transferred to second problem	Enhances performance, Rapid progress in training of second problem	Works with similar problems only

⋮

- 수많은 기술이 모여서 성능을 끌어올린 분야!
- 따라서 단순 모델공부만으로는 성능 향상이 힘들

4.2 딥러닝의 구성요소: 손실함수

1. 크로스엔트로피 손실함수 (Crossentropy function)

- 분류 문제에 맞게 고안된 목적 함수 (손실 함수, 에러 함수...etc)
- 엔트로피(entropy): information theorem(정보 이론) 에서 놀람도를 의미

Ex) 주사위의 각 눈이 나올 확률은 1/6 전혀 놀라운 결과가 아님 (놀람도 낮음)

>> 하지만 내가 모델링한 주사위 3 눈이 1/2로 나온다면 놀람도가 커지고 이는 모델링에 어려가 있음을 암시.

<직관적 설명>

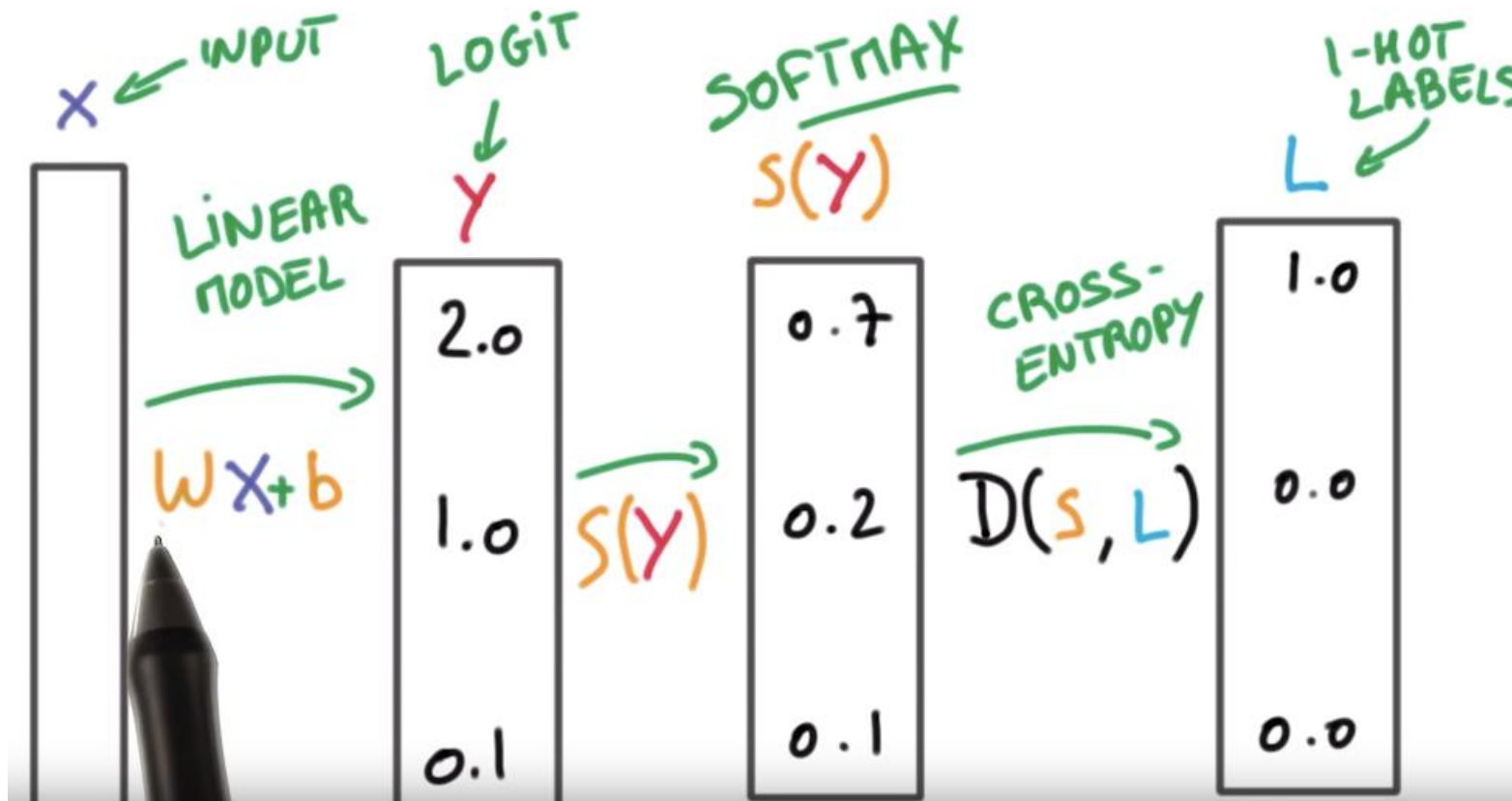
- 모델링 결과의 엔트로피를 낮춘다는 것은 어떤 모델이 학습을 통해 목적함수와 같은 output을 산출하여 놀람도를 낮춘다는 것과 마찬가지.

$$H(P, Q) = -\mathbb{E}_{X \sim P}[\log Q(x)] = -\sum P(x) \log Q(x)$$

P: 올바른 확률분포 / Q: 내가 예측한 확률분포

4.2 딥러닝의 구성요소: 손실함수

1. 크로스엔트로피 손실함수 (Crossentropy function)



- $S(Y)$ 분포(모델의 예측 결과) 와 L 분포(실제 정답)가 최대한 유사해야 함

4.2 딥러닝의 구성요소: 손실함수

1. 크로스엔트로피 손실함수 계산 예시(Crossentropy function)

$$\text{softmax(input)} = \begin{bmatrix} 0.2 \\ 0.7 \\ 0.1 \end{bmatrix} \Rightarrow \begin{matrix} Q(X = c_1) = 0.2 \\ Q(X = c_2) = 0.7 \\ Q(X = c_3) = 0.1 \end{matrix} \Rightarrow \begin{aligned} H(P, Q) &= - \sum P(x) \log Q(x) \\ &= -(0 \cdot \log 0.2 + 1 \cdot \log 0.7 + 0 \cdot \log 0.1) \\ &= -\log 0.7 \approx 0.357 \end{aligned}$$

1. 모델의 예측값 (Q 분포)

2. 정답분포(P)와의 비교

$$\begin{aligned} D_{\text{KL}}(p \parallel q) &= \sum_x p(x) \log \frac{p(x)}{q(x)} \\ &= \sum_x p(x) \log p(x) - \sum_x p(x) \log q(x) \\ &= -H(p) + \text{CE}(p, q) \end{aligned}$$

- 분류 문제에서 Cross-entropy는 두 분포의 Kullback-Leibler divergence를 최소화하는 것과 같은 문제

Recommended Reference:
<https://3months.tistory.com/436>
<https://theeluwin.postype.com/post/6080524>

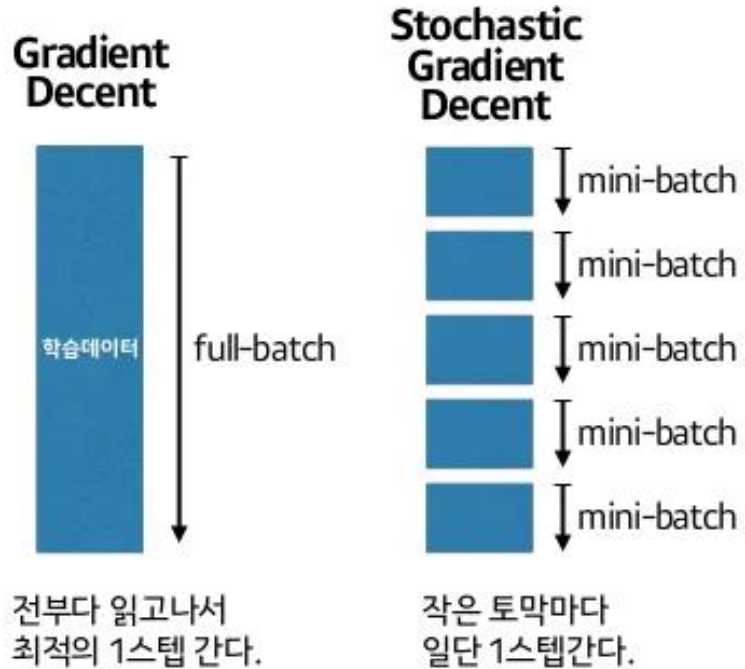
4.2 딥러닝의 구성요소: 손실함수

2. Binary Crossentropy function

$$l = -(y \log(p) + (1 - y) \log(1 - p))$$

- 이진분류 문제의 경우
- 모델의 마지막 node의 output(예측값): p
- Label (정답) : y (0 또는 1)
- 정답이 0이면 p 값이 0에 가까워져야 loss가 최소, 반대로 마찬가지로 >> 직접 숫자를 넣어보자

4.2 딥러닝의 구성요소 : Batch size



(Full) Batch training

- 학습을 통해 한번 모델이 업데이트 될 때 데이터 전체를 가지고 모델의 가중치를 업데이트하는 과정

Mini-batch training

- 학습을 통해 한번 모델이 업데이트 될 때 데이터 일부를 가지고 모델의 가중치를 업데이트하는 과정

4.2 딥러닝의 구성요소 : Batch size

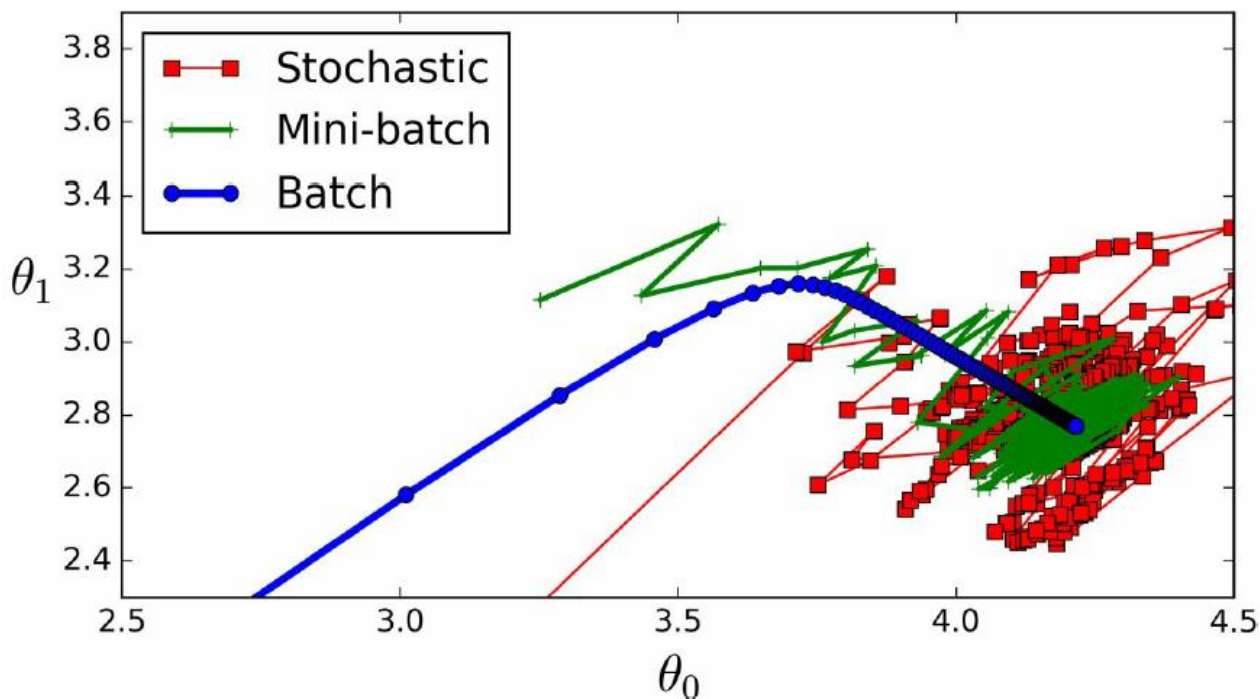
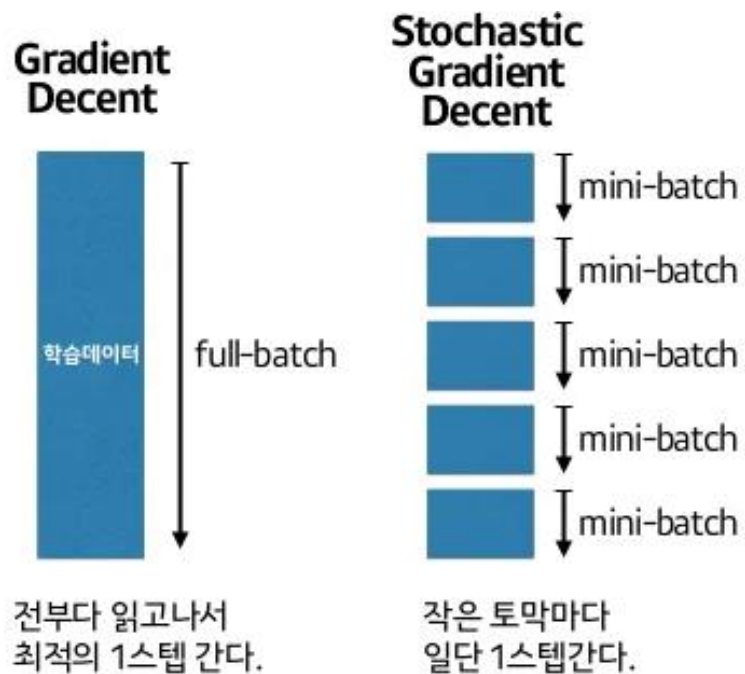
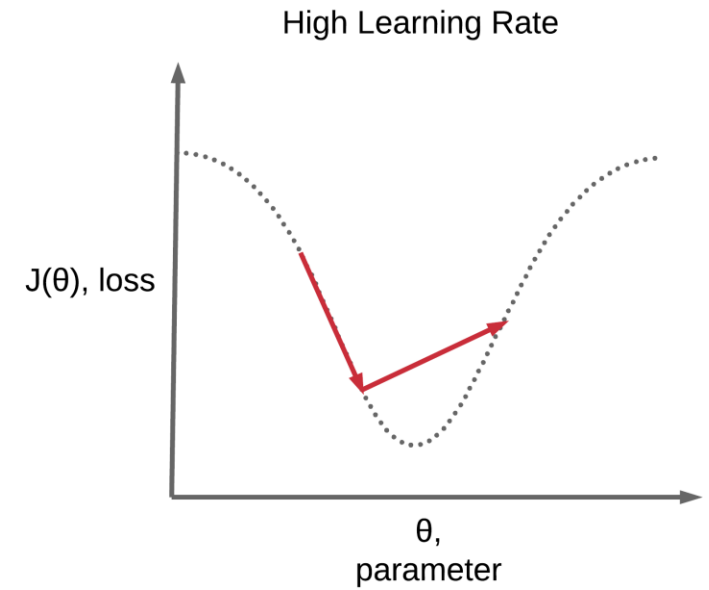
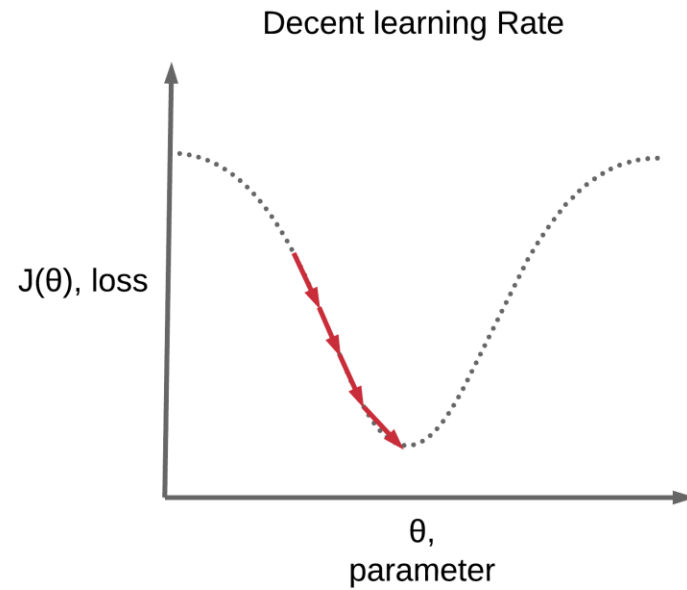
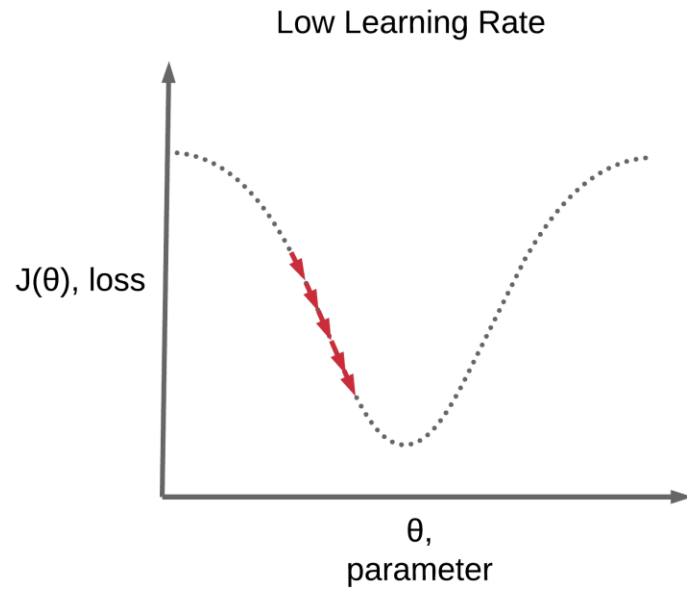


Figure 4-11. Gradient Descent paths in parameter space

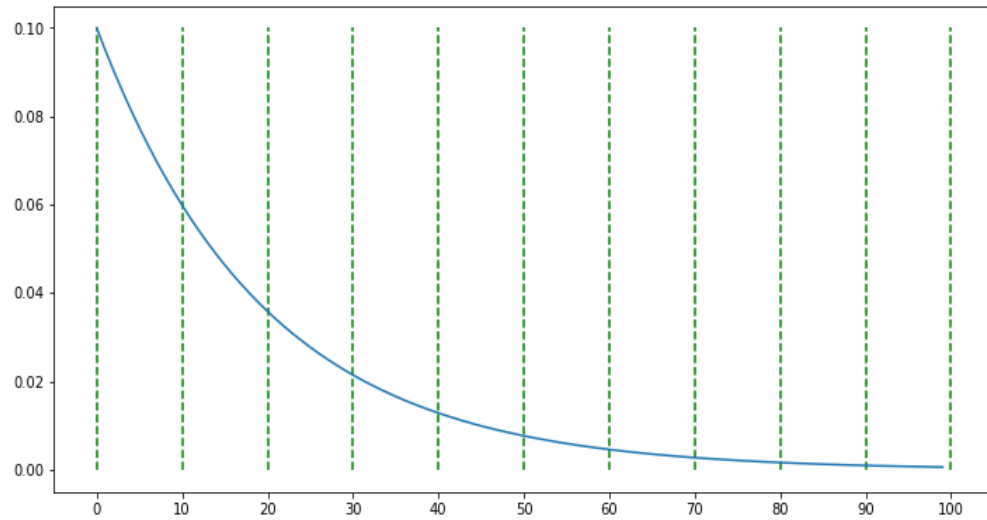
Full batch vs Mini-batch vs one-data

4.2 딥러닝의 구성요소 : Learning rate

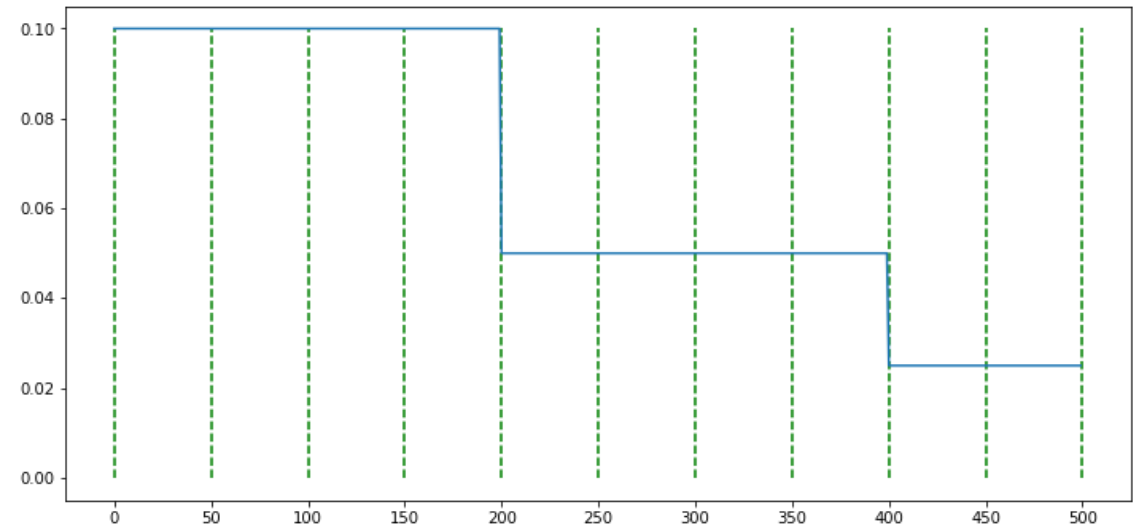


학습률을 어떻게 조정하는게 좋을까?

4.2 딥러닝의 구성요소 : Learning rate

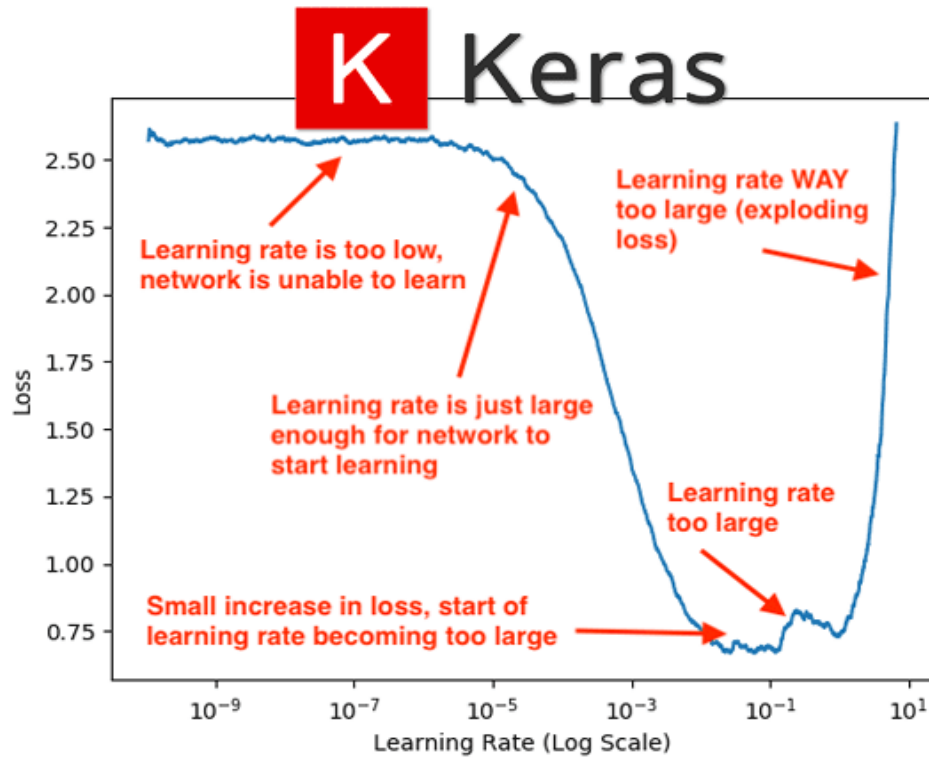


Lambda LR scheduler



Step LR scheduler

4.2 딥러닝의 구성요소 : Learning rate



Learning rate scheduler:

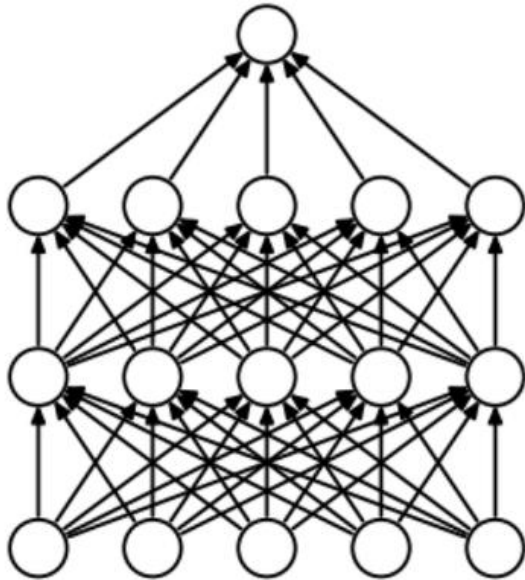
- 학습시 속도를 초반에는 빠르게, 학습이 어느 정도 진행된 이후에는 느리게 만들어주는 tool.
- `keras.callbacks.LearningRateScheduler(schedule, verbose=0)`

4.3 딥러닝 모델의 성능 향상 방법

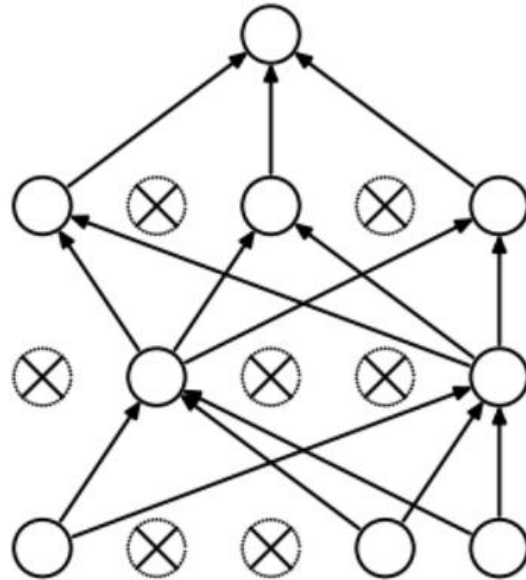
1. Dropout
2. Batch normalization
3. 과적합 방지
4. ...

4.3 딥러닝 모델의 성능 향상 방법

1. Dropout



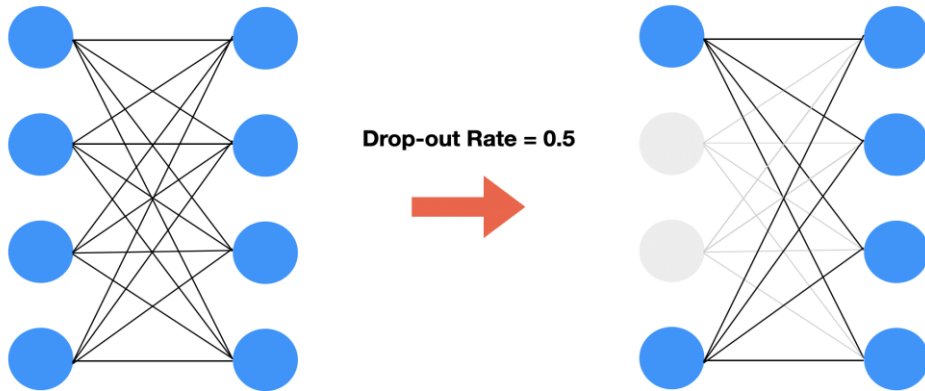
(a) Standard Neural Net



(b) After applying dropout.

- Dropout은 신경망의 뉴런을 부분적으로 생략하여 모델의 과적합(overfitting)을 해결해 주기 위한 방법.
- 과적합이란? >> 모델이 학습 데이터만 정답을 잘 맞추고 테스트 데이터에서 성능이 떨어지는 현상
- 학습할때 계속 연결되는 node를 바꿈으로써 특정 weight에 집중되는 현상을 막음.

4.3 딥러닝 모델의 성능 향상 방법



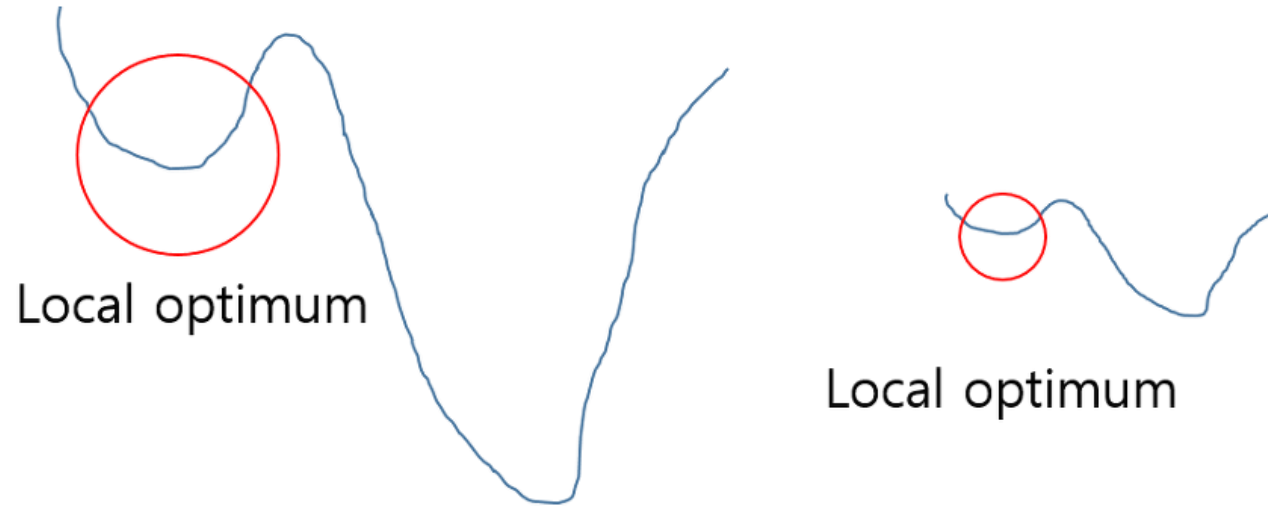
<https://heytech.tistory.com/>

Dropout layer:

- Drop-out은 어떤 특정한 Feature만을 과도하게 집중하여 학습함으로써 발생할 수 있는 과대적합(Overfitting)을 방지하기 위해 사용.
- `model.add(tf.keras.layers.Dropout(drop_rate))`

4.3 딥러닝 모델의 성능 향상 방법

2. Batch normalization



- (좌) Normalization 적용 전 / (우) Normalization 적용 후

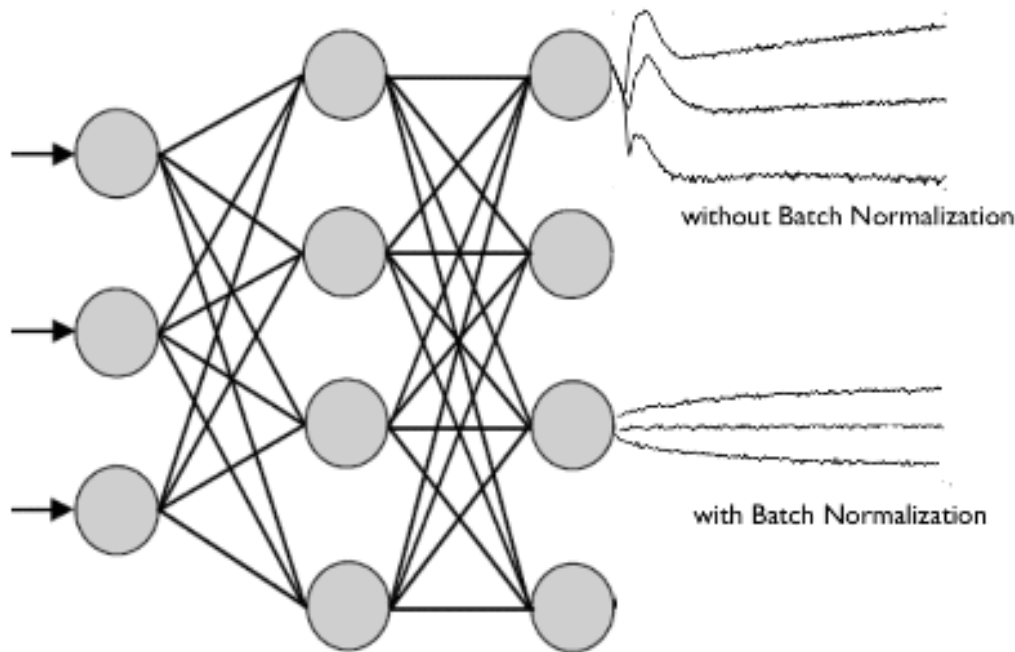
정규화를 하는 이유:

1) 빠른 학습

2) Local optimum 문제에 빠지는 가능성을 줄임.

4.3 딥러닝 모델의 성능 향상 방법

2. Batch normalization



- 배치 정규화 논문에서는 학습에서 불안정화가 일어나는 이유를 'Internal Covariance Shift' 라고 주장. 이는 네트워크의 각 레이어나 Activation 마다 입력값의 분산이 달라지는 현상을 뜻함.
- Covariate Shift : 이전 레이어의 파라미터 변화로 인하여 현재 레이어의 입력의 분포가 바뀌는 현상.
- Internal Covariate Shift : 레이어를 통과할 때 마다 Covariate Shift 가 일어나면서 입력의 분포가 약간씩 변하는 현상.

4.3 딥러닝 모델의 성능 향상 방법

3. 과적합 방지

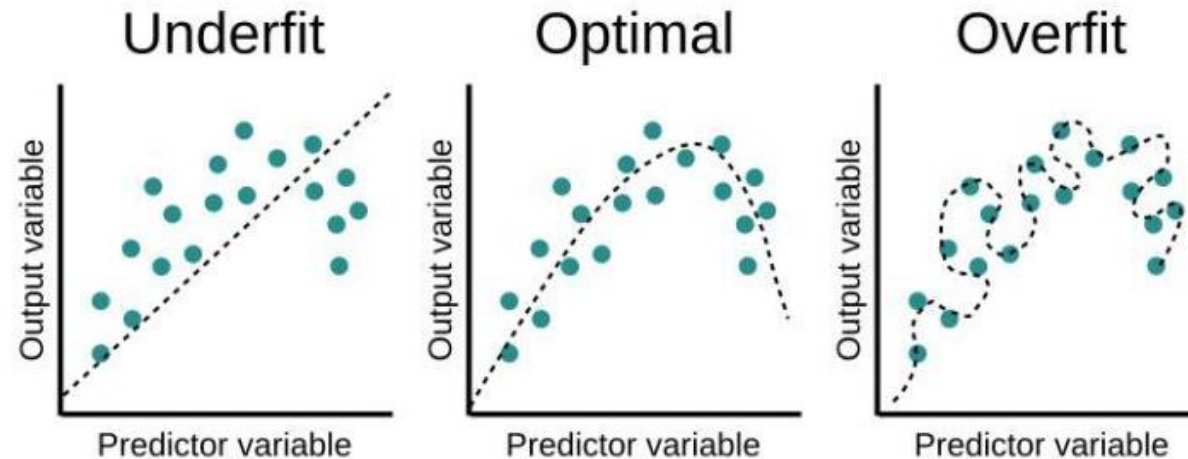
과적합의 이해

- 과대적합 (Overfitting)

학습 데이터에 대해서만 너무 학습이 잘 되어서 테스트 데이터에 대한 성능이 떨어지는 단계

- 과소적합 (Underfitting)

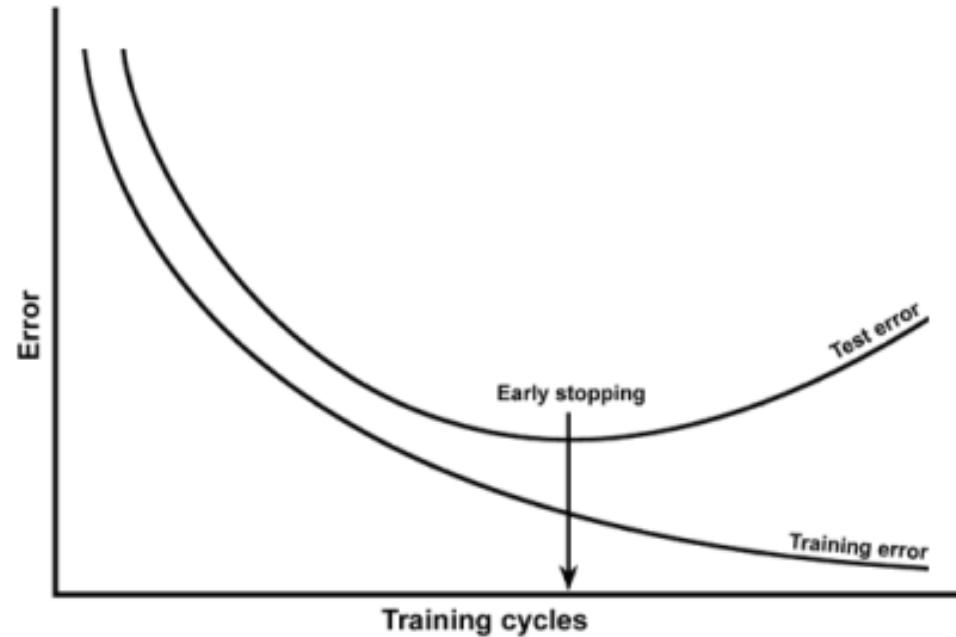
학습이 덜 이루어지는 단계



4.3 딥러닝 모델의 성능 향상 방법

3. 과적합 방지

과적합의 확인



- Train loss는 계속 감소하는데 Test set의 loss가 증가하는 경우
- 혹은 Train accuracy는 향상하는데 Test accuracy가 감소하는 경우

4.3 딥러닝 모델의 성능 향상 방법

3. 과적합 방지

과적합의 방지 방법

0. Early stopping 적용

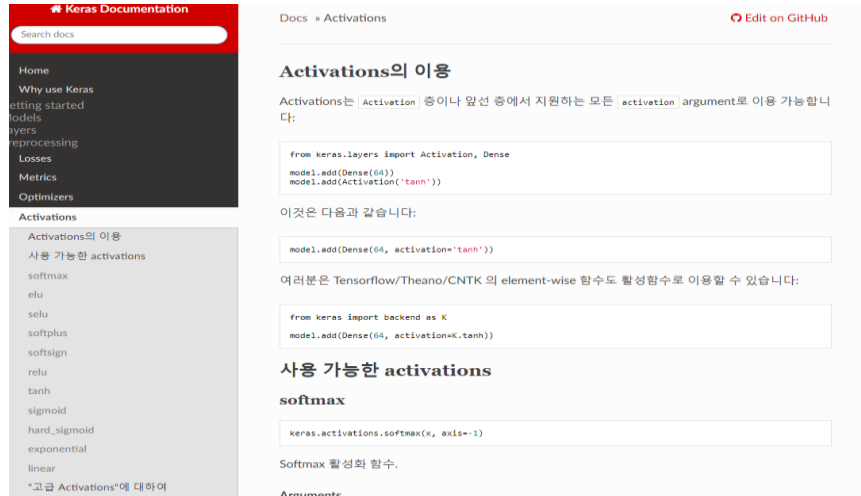
1. 데이터 양 늘리기 (Data augmentation)

2. 모델 복잡도 줄이기

3. 가중치 규제 적용

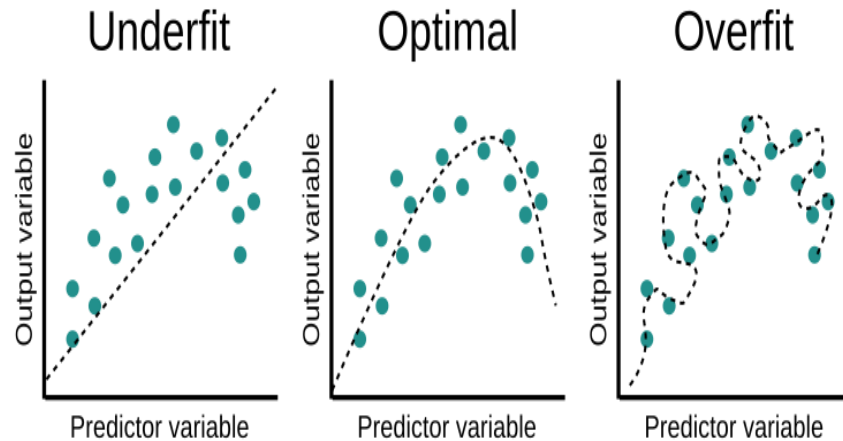
4. Dropout, normalization 등의 방법 사용

4.3 딥러닝 모델의 성능 향상 방법



Activation function:

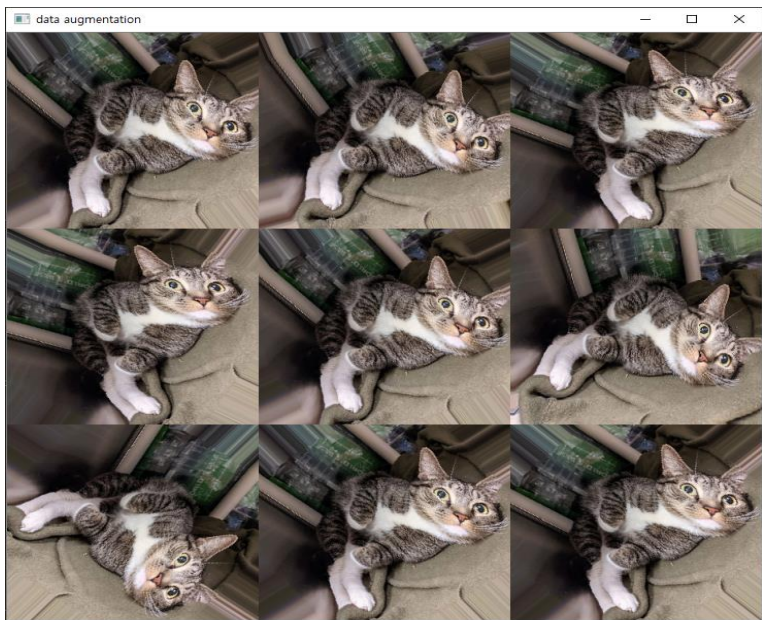
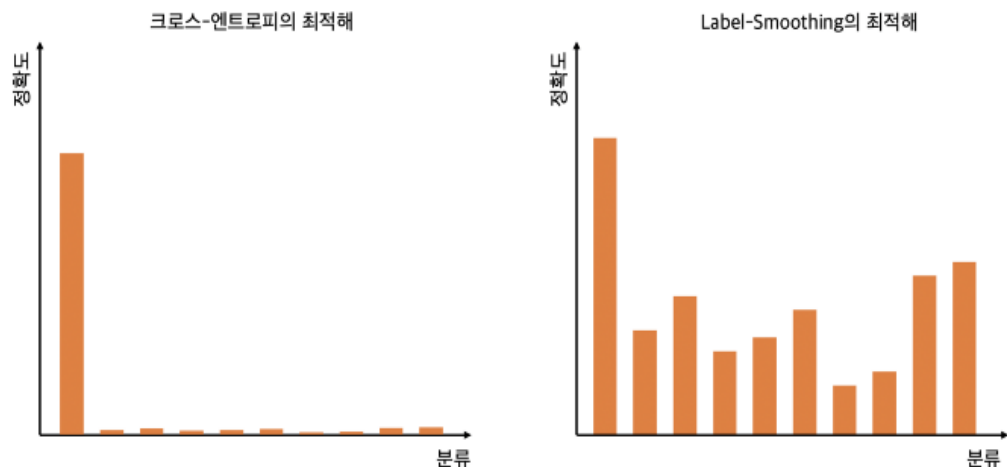
- GeLu, Leaky relu 등 다양한 발전 모델
- Keras activation



Weight decay:

- weight 값을 일정 범위 안에 고정시켜 과적합 방지.
- `layer = layers.Dense(...
kernel_regularizer=regularizers.L1L2(l1=1e-5, l2=1e-4),
bias_regularizer=regularizers.L2(1e-4),
activity_regularizer=regularizers.L2(1e-5))`

4.3 딥러닝 모델의 성능 향상 방법



Label smoothing:

- 과적합(Overfitting) 방지: 딥러닝 모델이 훈련 데이터에 지나치게 적응되어 새로운 데이터에 대한 일반화 능력이 저하되는 것을 방지. 훈련 데이터에 있는 라벨에 대해 완벽하게 맞추려고 하는 모델은 높은 복잡성을 가질 수 있으며, 이는 새로운 데이터에 대한 예측을 어렵게 만듭니다.
- Soft Decision Boundary: 라벨 스무딩은 모델이 클래스 예측을 더 부드럽게 만들어주어, 모델의 결정 경계가 더 매끄럽고 일반적인 패턴을 학습하게 합니다. 이는 더 좋은 일반화 능력을 가져올 수 있습니다.
- `tf.keras.losses.CategoricalCrossentropy(from_logits=False, label_smoothing=0.0...)`

Data augmentation:

- 이미지 데이터 augmentation을 통해 학습 데이터를 늘려 주는 방법
- `keras.preprocessing.image.ImageDataGenerator(featurewise_center=False, ...width_shift_range=0.0, height_shift_range=0.0, brightness_range=None, shear_range=0.0, zoom_range=0.0, ..., cval=0.0, horizontal_flip=False, vertical_flip=False, rescale=None, preprocessing_function=None, data_format=None, validation_split=0.0, dtype=None)`

감사합니다